CONSTRAINT-BASED PROBLEM SOLVING

University Examination Scheduling using an Intelligent Agent

Given Problem

A university has to schedule final examinations for five courses:

- A – Artificial Intelligence
- B – Database Systems
- C – Computer Networks
- D – Operating Systems
- E – Software Engineering

There are three available examination slots:

- Slot 1
- Slot 2
- Slot 3

The conflicting course pairs that must not be assigned to the same slot are:

A–B, A–C, B–C, B–D, C–D, C–E, D–E.

The objective is to generate a conflict-free timetable satisfying all constraints.

a) PEAS Description of the Exam-Scheduling Agent

Introduction

PEAS stands for:

- **P – Performance Measure**
- **E – Environment**
- **A – Actuators**
- **S – Sensors**

The exam-scheduling agent is an intelligent system that receives information about courses,

available slots and conflicts, and generates a valid examination timetable.

Component	Description
Performance Measure	Produce a conflict-free timetable, satisfy all course constraints, use the available slots effectively, avoid clashes between courses having common students, and generate a feasible schedule.
Environment	University examination system consisting of 5 courses, 3 examination slots, student enrollment information and conflict relationships between courses.
Actuators	Assign courses to examination slots, modify assignments, remove conflicting assignments, and publish the final timetable.
Sensors	Course information, student enrollment data, available examination slots, conflict pairs and current timetable assignments.

1. Performance Measure

The agent should:

1. Generate a conflict-free examination timetable.
2. Ensure every course receives exactly one slot.
3. Ensure conflicting courses are assigned to different slots.
4. Use only the available three slots.
5. Produce a valid and feasible timetable efficiently.

2. Environment

The environment consists of:

- 5 courses: A, B, C, D and E.
- 3 available slots.
- Student enrollment relationships.
- Seven conflict constraints.
- University examination scheduling requirements.

3. Actuators

The agent can:

- Assign a course to Slot 1, Slot 2 or Slot 3.
- Change an existing assignment.
- Backtrack when a conflict occurs.
- Produce the final examination timetable.

4. Sensors

The agent obtains:

- Course list.
- Available slots.
- Student enrollment information.
- Conflict pairs.
- Current assignments.
- Constraint violations.

Python code:

```
# PEAS Description for Exam Scheduling Agent

peas = {
    "Performance Measure": [
        "Generate a conflict-free timetable",
        "Assign every course to exactly one slot",
        "Avoid clashes between conflicting courses",
        "Use available examination slots effectively"
    ],

    "Environment": [
        "5 university courses",
        "3 examination slots",
        "Student enrollment information",
        "Course conflict information"
    ],

    "Actuators": [
        "Assign courses to examination slots",
        "Change course assignments",
        "Generate the final timetable"
    ],

    "Sensors": [
        "Course information",
        "Available slots",
        "Student enrollment data",
        "Conflict pairs"
    ]
}

print("=" * 55)
print("PEAS DESCRIPTION - EXAM SCHEDULING AGENT")
print("=" * 55)

for component, details in peas.items():
    print("\n" + component + ":")
    for item in details:
        print(" -", item)
```

Output:

```
============================================================
PEAS DESCRIPTION - EXAM SCHEDULING AGENT
============================================================

Performance Measure:
 - Generate a conflict-free timetable
 - Assign every course to exactly one slot
 - Avoid clashes between conflicting courses
 - Use available examination slots effectively

Environment:
 - 5 university courses
 - 3 examination slots
 - Student enrollment information
 - Course conflict information

Actuators:
 - Assign courses to examination slots
 - Change course assignments
 - Generate the final timetable

Sensors:
 - Course information
 - Available slots
 - Student enrollment data
 - Conflict pairs
```

b) Type of Intelligent Agent

The four possible agent types are:

1. **Simple Reflex Agent**
2. **Model-Based Agent**
3. **Goal-Based Agent**
4. **Utility-Based Agent**

For this examination scheduling problem, the Goal-Based Agent is the most appropriate.

Goal-Based Agent

The primary objective of the agent is to achieve a specific goal:

Generate a valid examination timetable in which no conflicting courses are

assigned to the same slot.

The agent must consider the current assignments and determine which future assignments will lead to the desired goal.

Characteristics

A goal-based scheduling agent:

- **Has a clearly defined goal.**
- **Considers possible actions before making assignments.**
- **Checks whether constraints are satisfied.**
- **Can search through different possible schedules.**
- **Uses backtracking when an assignment leads to an invalid state.**
- **Stops when it reaches a valid complete timetable.**

Not Simple Reflex Agent

A simple reflex agent reacts only to the current situation using condition-action rules.

For example:

"If there is a conflict, change the slot."

This is insufficient because scheduling requires consideration of future assignments and constraints.

Why Not Model-Based Agent?

A model-based agent maintains an internal representation of the environment. Although this can help, the main requirement here is to achieve a specific scheduling goal, making goal-based reasoning more suitable.

Why Not Utility-Based Agent?

A utility-based agent is useful when several valid solutions exist and the system must choose the best solution based on preferences such as:

- minimum examination days,
- balanced workload,
- preferred slots,

- room utilization.

However, the given problem only asks for a conflict-free timetable, not optimization among multiple valid timetables.

Final Answer

Therefore:

> The exam-scheduling system is best classified as a Goal-Based Intelligent Agent because it searches for a timetable that satisfies the defined scheduling goal and all constraints.

Python code:

```
# Goal-Based Agent for Exam Scheduling

courses = ["A", "B", "C", "D", "E"]
slots = ["Slot1", "Slot2", "Slot3"]

# Conflicting course pairs
conflicts = [
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("C", "D"),
    ("C", "E"),
    ("D", "E")
]

# Goal: create a conflict-free timetable
timetable = {
    "A": "Slot1",
    "B": "Slot2",
    "C": "Slot3",
    "D": "Slot1",
    "E": "Slot2"
}

def check_goal(timetable):
    for course1, course2 in conflicts:
        if timetable[course1] == timetable[course2]:
            return False
    return True

print("=" * 55)
print("GOAL-BASED EXAM SCHEDULING AGENT")
print("=" * 55)

print("\nTimetable:")
for course, slot in timetable.items():
    print(course, "->", slot)

print("\nGoal: Generate a conflict-free timetable")

if check_goal(timetable):
    print("Goal Status: ACHIEVED")
    print("Result: Timetable is conflict-free.")
else:
    print("Goal Status: NOT ACHIEVED")
    print("Result: Timetable contains conflicts.")
```

Output

```
------------------------------------------------------------
============================================================
GOAL-BASED EXAM SCHEDULING AGENT
============================================================

Timetable:
A -> Slot1
B -> Slot2
C -> Slot3
D -> Slot1
E -> Slot2

Goal: Generate a conflict-free timetable
Goal Status: ACHIEVED
Result: Timetable is conflict-free.
```

c) CSP Formulation and Constraint Graph

A Constraint Satisfaction Problem (CSP) consists of:

1. **Variables**
2. **Domains**
3. **Constraints**

The examination scheduling problem can be represented as a CSP.

1. Variables

Each course represents one variable.

X={A,B,C,D,E}X = \{A,B,C,D,E\}

Where:

- A = Artificial Intelligence
- B = Database Systems
- C = Computer Networks
- D = Operating Systems

- E = Software Engineering

2. Domains

Each course can be assigned one of the three examination slots.

D(A)=D(B)=D(C)=D(D)=D(E)D(A)=D(B)=D(C)=D(D)=D(E) ={Slot1,Slot2,Slot3}

Variable	Domain
A	{S1, S2, S3}
B	{S1, S2, S3}
C	{S1, S2, S3}
D	{S1, S2, S3}
E	{S1, S2, S3}

3. Constraints

Two conflicting courses cannot be assigned to the same slot.

Therefore:

A≠BA \neq B A≠CA \neq C B≠CB \neq C B≠DB \neq D C≠DC \neq D C≠EC \neq E D≠ED \neq E

These are binary inequality constraints.

The given conflict pairs are specified in the assessment scenario.

Constraint Graph

In a constraint graph:

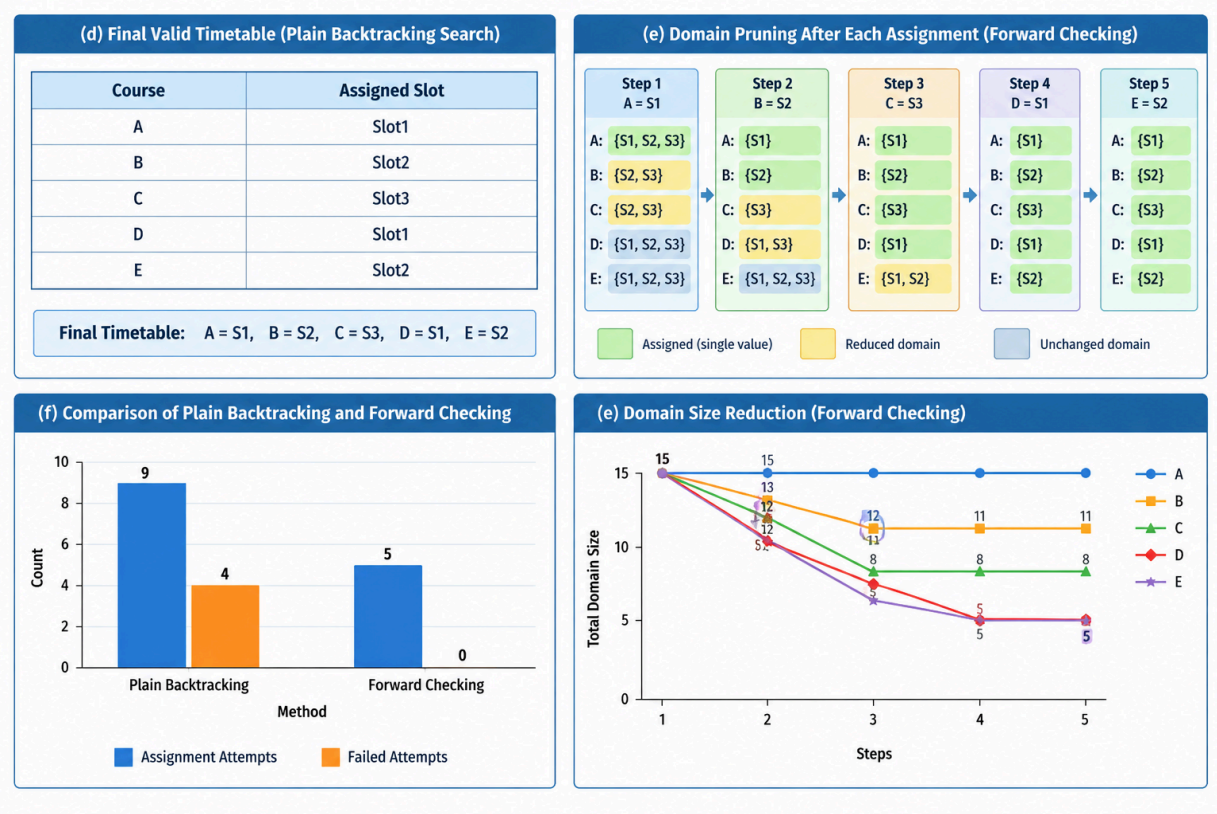
- Vertices represent variables/courses.
- Edges represent conflicts/constraints.

Vertices

V={A,B,C,D,E}V=\{A,B,C,D,E\}

Edges

E={(A,B),(A,C),(B,C),(B,D),(C,D),(C,E),(D,E)}



Course	Conflicting Courses
A	B, C
B	A, C, D
C	A, B, D, E
D	B, C, E
E	C, D

Notice that C has the highest degree, with four conflicts. This makes C highly constrained.

CSP Formulation

Therefore, the complete CSP is:

CSP=(X,D,C)CSP=(X,D,C)

where:

X={A,B,C,D,E}X=\{A,B,C,D,E\} D={S1,S2,S3}D=\{S1,S2,S3\}

and

C={A=B,A=C,B=C,B=D,C=D,C=E,D=E}

d) Plain Backtracking Search

The assessment specifically requires:

- Static variable order: A → B → C → D → E
- Domain order: Slot1 → Slot2 → Slot3
- Every assignment attempt.
- Failed attempts.
- Backtracking steps.
- Final timetable.

Backtracking Principle

Plain backtracking assigns values to variables one at a time.

If an assignment violates a constraint, that value is rejected and the algorithm tries the next available value.

The search order is:

A→B→C→D→EA\rightarrow B\rightarrow C\rightarrow D\rightarrow E

Value order:

S1→S2→S3S1\rightarrow S2\rightarrow S3

Step-by-Step Search

Step 1: Assign A

Try:

A=S1

Step 2: Assign B

Try first value:

B=S1B=S1

But A and B conflict:

A≠BA\neq B

Since:

A=S1, B=S1A=S1,\ B=S1

the assignment fails.

Backtrack

Try next value:

B=S2B=S2

This satisfies:

A=B

Step 3: Assign C

Try first value:

C=S1C=S1

But:

A≠CA\neq C

and A is already S1.

Try:

C=S3C=S3

Now:

A=S1, B=S2, C=S3A=S1,\ B=S2,\ C=S3

All constraints among A, B and C are satisfied.

Step 4: Assign D

D conflicts with B and C:

D≠BD\neq B D≠CD\neq C

Try:

D=S1D=S1

B = S2 and C = S3, so D = S1 does not conflict with either.

Step 5: Assign E

E conflicts with C and D:

E≠CE\neq C E≠DE\neq D

Try:

E=S1E=S1

But:

D=S1D=S1

and D–E is a conflict.

Try next value:

E=S2E=S2

C = S3 and D = S1, therefore E = S2 creates no conflict.

All five variables are now assigned.

Step	Variable	Value Tried	Current Assignment	Result	Action
1	A	S1	A=S1	✓	Accept
2	B	S1	A=S1, B=S1	✗ A-B conflict	Try next value
3	B	S2	A=S1, B=S2	✓	Accept
4	C	S1	A=S1, B=S2, C=S1	✗ A-C conflict	Try next value
5	C	S2	A=S1, B=S2, C=S2	✗ B-C conflict	Try next value
6	C	S3	A=S1, B=S2, C=S3	✓	Accept
7	D	S1	A=S1, B=S2, C=S3, D=S1	✓	Accept
8	E	S1	A=S1, B=S2, C=S3, D=S1, E=S1	✗ D-E conflict	Try next value
9	E	S2	A=S1, B=S2, C=S3, D=S1, E=S2	✓	Accept

Examination Slot	Course
Slot 1	A – Artificial Intelligence
Slot 2	B – Database Systems
Slot 3	C – Computer Networks
Slot 1	D – Operating Systems
Slot 2	E – Software Engineering

Final Assignment

A=S1, B=S2, C=S3, D=S1, E=S2\boxed{A=S1,\ B=S2,\ C=S3,\ D=S1,\ E=S2}

Verification

- A–B → S1 ≠ S2 ✓
- A–C → S1 ≠ S3 ✓
- B–C → S2 ≠ S3 ✓
- B–D → S2 ≠ S1 ✓
- C–D → S3 ≠ S1 ✓
- C–E → S3 ≠ S2 ✓
- D–E → S1 ≠ S2 ✓

Therefore, the timetable is completely conflict-free.

Python code:

```
# Plain Backtracking Search for Exam Scheduling

courses = ["A", "B", "C", "D", "E"]
slots = ["Slot1", "Slot2", "Slot3"]

# Conflicting course pairs
conflicts = [
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("C", "D"),
    ("C", "E"),
    ("D", "E")
]

assignment = {}
attempts = 0
failures = 0


def is_valid(course, slot):
    """Check whether assigning slot to course creates a conflict."""
    for c1, c2 in conflicts:

        if c1 == course and c2 in assignment:
            if assignment[c2] == slot:
                return False

        if c2 == course and c1 in assignment:
            if assignment[c1] == slot:
                return False

    return True


def backtracking(index):
    global attempts, failures

    # All courses assigned
    if index == len(courses):
        return True

    course = courses[index]

    # Try Slot1 -> Slot2 -> Slot3
    for slot in slots:
        attempts += 1

        print(f"Attempt {attempts}: {course} = {slot}")

        if is_valid(course, slot):
            assignment[course] = slot

            print("  Accepted")
            print("  Current assignment:", assignment)
```

```
            if backtracking(index + 1):
                return True

            # Backtracking
            print(f"  Backtracking from {course} = {slot}")
            del assignment[course]

        else:
            failures += 1
            print("  Rejected - Conflict detected")

    return False


print("=" * 60)
print("PLAIN BACKTRACKING SEARCH")
print("=" * 60)

if backtracking(0):

    print("\n" + "=" * 60)
    print("FINAL VALID TIMETABLE")
    print("=" * 60)

    for course in courses:
        print(course, "->", assignment[course])

    print("\nTotal assignment attempts:", attempts)
    print("Failed attempts:", failures)

else:
    print("No valid timetable found.")
```

Output:

```
============================================================
PLAIN BACKTRACKING SEARCH
============================================================
Attempt 1: A = Slot1
  Accepted
  Current assignment: {'A': 'Slot1'}
Attempt 2: B = Slot1
  Rejected - Conflict detected
Attempt 3: B = Slot2
  Accepted
  Current assignment: {'A': 'Slot1', 'B': 'Slot2'}
Attempt 4: C = Slot1
  Rejected - Conflict detected
Attempt 5: C = Slot2
  Rejected - Conflict detected
Attempt 6: C = Slot3
  Accepted
  Current assignment: {'A': 'Slot1', 'B': 'Slot2', 'C': 'Slot3'}
Attempt 7: D = Slot1
  Accepted
  Current assignment: {'A': 'Slot1', 'B': 'Slot2', 'C': 'Slot3', 'D': 'Slot1'}
Attempt 8: E = Slot1
  Rejected - Conflict detected
Attempt 9: E = Slot2
  Accepted
  Current assignment: {'A': 'Slot1', 'B': 'Slot2', 'C': 'Slot3', 'D': 'Slot1', 'E': 'Slot2'}

============================================================
FINAL VALID TIMETABLE
============================================================
A -> Slot1
B -> Slot2
C -> Slot3
D -> Slot1
E -> Slot2

Total assignment attempts: 9
Failed attempts: 4
```

e) Backtracking with Forward Checking

20 Marks

Forward checking improves ordinary backtracking by immediately removing inconsistent values from the domains of unassigned variables after every assignment.

The same variable and value order is used:

A→B→C→D→EA\rightarrow B\rightarrow C\rightarrow D\rightarrow E

and:

S1→S2→S3

Variable	Initial Domain
A	{S1, S2, S3}

B	{S1, S2, S3}
C	{S1, S2, S3}
D	{S1, S2, S3}
E	{S1, S2, S3}

Step 1: Assign A = S1

Since A conflicts with B and C, remove S1 from their domains.

Before:

B = {S1, S2, S3}
C = {S1, S2, S3}

After forward checking:

B = {S2, S3}
C = {S2, S3}

D and E are not directly connected to A, so their domains remain unchanged.

Domains after A=S1

Variable	**Remaining Domain**
A	{S1}
B	{S2, S3}
C	{S2, S3}
D	{S1, S2, S3}
E	{S1, S2, S3}

Variable	**Remaining Domain**

A	{S1}
B	{S2, S3}
C	{S2, S3}
D	{S1, S2, S3}
E	{S1, S2, S3}

Step 2: Assign B = S2

B's domain is:

{S2,S3}\{S2,S3\}

Following the specified value order, choose:

B=S2B=S2

B conflicts with C and D.

Therefore remove S2 from their domains.

C:

{S2,S3}→{S3}\{S2,S3\}\rightarrow\{S3\}

D:

{S1,S2,S3}→{S1,S3}\{S1,S2,S3\}\rightarrow\{S1,S3\}

Domains after B=S2

Variable	**Remaining Domain**
A	{S1}
B	{S2}
C	{S3}
D	{S1, S3}

E	{S1, S2, S3}

Notice that forward checking has already discovered that:

C=S3C=S3

is forced.

Step 3: Assign C = S3

C has only one remaining value:

C=S3C=S3

C conflicts with D and E.

Therefore remove S3 from their domains.

D:

{S1,S3}→{S1}\{S1,S3\}\rightarrow\{S1\}

E:

{S1,S2,S3}→{S1,S2}\{S1,S2,S3\}\rightarrow\{S1,S2\}

Domains after C=S3

Variable	**Remaining Domain**
A	{S1}
B	{S2}
C	{S3}
D	{S1}
E	{S1, S2}

D is now forced to S1.

Step 4: Assign D = S1

D has only:

D=S1D=S1

D conflicts with E.

Therefore remove S1 from E's domain.

E={S1,S2}E=\{S1,S2\}

becomes:

E={S2}E=\{S2\}

Domains after D=S1

Variable	Remaining Domain
A	{S1}
B	{S2}
C	{S3}
D	{S1}
E	{S2}

Therefore E is forced to S2.

Step 5: Assign E = S2

E=S2E=S2

There are no remaining conflicts.

Therefore the complete assignment is:

A=S1, B=S2, C=S3, D=S1, E=S2\boxed{A=S1,\ B=S2,\ C=S3,\ D=S1,\ E=S2}

Forward Checking Table

Step	Assignment	Domain Pruning	Remaining Important Domains
1	A=S1	Remove S1 from B,C	B={S2,S3}, C={S2,S3}
2	B=S2	Remove S2 from C,D	C={S3}, D={S1,S3}
3	C=S3	Remove S3 from D,E	D={S1}, E={S1,S2}
4	D=S1	Remove S1 from E	E={S2}
5	E=S2	No values left to prune	Complete solution

Final Timetable Using Forward Checking

Slot	Course
Slot 1	A – Artificial Intelligence
Slot 1	D – Operating Systems
Slot 2	B – Database Systems
Slot 2	E – Software Engineering
Slot 3	C – Computer Networks

Python code:

```
courses = ['A','B','C','D','E']
slots = ['S1','S2','S3']

conflicts = [('A','B'),('A','C'),('B','C'),
             ('B','D'),('C','D'),('C','E'),('D','E')]

domains = {c: slots[:] for c in courses}
answer = {}

def forward(c, s):
    for x, y in conflicts:
        if x == c and y in domains:
            domains[y] = [v for v in domains[y] if v != s]
        if y == c and x in domains:
            domains[x] = [v for v in domains[x] if v != s]

def solve(i=0):
    if i == len(courses):
        return True

    c = courses[i]

    for s in domains[c][:]:
        ok = True

        for x, y in conflicts:
            if (x == c and answer.get(y) == s) or \
               (y == c and answer.get(x) == s):
                ok = False

        if ok:
            old = {x: domains[x][:] for x in courses}
            answer[c] = s

            forward(c, s)

            if all(domains[x] for x in courses):
                print(c, "=", s, " Domains:", domains)
                if solve(i + 1):
                    return True

            domains.update(old)
            del answer[c]

    return False

solve()

print("\nFinal Timetable:")
for c in courses:
    print(c, "->", answer[c])
```

Output:

```
A = S1  Domains: {'A': ['S1', 'S2', 'S3'], 'B': ['S2', 'S3'], 'C': ['S2', 'S3'], 'D': ['S1', 'S2', 'S3'], 'E': ['S1', 'S2', 'S3']}
B = S2  Domains: {'A': ['S1', 'S3'], 'B': ['S2', 'S3'], 'C': ['S3'], 'D': ['S1', 'S3'], 'E': ['S1', 'S2', 'S3']}
C = S3  Domains: {'A': ['S1'], 'B': ['S2'], 'C': ['S3'], 'D': ['S1'], 'E': ['S1', 'S2']}
D = S1  Domains: {'A': ['S1'], 'B': ['S2'], 'C': ['S3'], 'D': ['S1'], 'E': ['S2']}
E = S2  Domains: {'A': ['S1'], 'B': ['S2'], 'C': ['S3'], 'D': ['S1'], 'E': ['S2']}

Final Timetable:
A -> S1
B -> S2
C -> S3
D -> S1
E -> S2
```

Therefore:

A=S1, B=S2, C=S3, D=S1, E=S2\boxed{A=S1,\ B=S2,\ C=S3,\ D=S1,\ E=S2}

The final timetable is the same valid timetable obtained through plain backtracking.

f) Comparison: Plain Backtracking vs Forward Checking

Forward checking is more efficient because it detects potential conflicts before the algorithm reaches the corresponding variable assignment.

The assessment specifically asks for comparison based on assignment attempts, backtracks and overall efficiency.

Comparison Table

Feature	**Plain Backtracking**	**Backtracking + Forward Checking**
Variable order	A → B → C → D → E	A → B → C → D → E
Value order	S1 → S2 → S3	S1 → S2 → S3
Initial domains	All values available	All values available
Domain pruning	No	Yes
Failed value attempts	4	0
Successful assignments	5	5

Total value-assignment attempts	9	5
Backtracking/value changes	4	0
Early detection of conflicts	No	Yes
Search effort	Higher	Lower
Efficiency	Lower	Higher

Plain Backtracking Attempts

The unsuccessful attempts are:

1. B = S1 → A-B conflict
2. C = S1 → A-C conflict
3. C = S2 → B-C conflict
4. E = S1 → D-E conflict

Therefore:

9 total assignment attempts

Forward Checking Attempts

Forward checking removes inconsistent values before they are attempted.

For example, after:

A=S1A=S1

it immediately removes S1 from B and C.

Therefore the algorithm does not waste an assignment attempt on:

B=S1B=S1

Similarly, after:

B=S2B=S2

C's S2 is removed before C is assigned.

Thus:

C=S3C=S3

is selected directly.

The five actual assignments are:

A=S1A=S1 B=S2B=S2 C=S3C=S3 D=S1D=S1 E=S2E=S2

Hence:

5 assignment attempts\boxed{5\text{ assignment attempts}}

and:

0 value-level backtracks\boxed{0\text{ value-level backtracks}}

for this search path.

Why Forward Checking Reduces Search Effort

Forward checking reduces search effort because it looks ahead after each assignment.

For example:

Without Forward Checking

After assigning:

A=S1A=S1

plain backtracking later tries:

B=S1B=S1

and discovers the conflict.

With Forward Checking

Immediately after:

A=S1A=S1

the algorithm removes S1 from B's domain:

B={S2,S3}B=\{S2,S3\}

Python code:

```
courses = ['A','B','C','D','E']
slots = ['S1','S2','S3']

conflicts = [('A','B'),('A','C'),('B','C'),
             ('B','D'),('C','D'),('C','E'),('D','E')]

# Results from the two methods
plain_attempts = 9
plain_failures = 4

forward_attempts = 5
forward_failures = 0

print("COMPARISON")
print("-" * 35)
print("Method              Attempts   Failures")
print("Plain Backtracking  ", plain_attempts, "       ", plain_failures)
print("Forward Checking    ", forward_attempts, "       ", forward_failures)

reduction = plain_attempts - forward_attempts
print("\nAssignment attempts reduced by:", reduction)

if forward_attempts < plain_attempts:
    print("Forward Checking is more efficient.")
    print("Reason: It removes invalid slot values before assignment.")
```

Output:

```
COMPARISON
-----------------------------------
Method              Attempts   Failures
Plain Backtracking   9         4
Forward Checking     5         0

Assignment attempts reduced by: 4
Forward Checking is more efficient.
Reason: It removes invalid slot values before assignment.
```

Therefore:

B=S1B=S1

is never attempted.

Similarly, after B=S2:

C={S3}C=\{S3\}

so the algorithm never tries C=S1 or C=S2.

This prevents unnecessary exploration of invalid branches.

Overall Analysis

Plain backtracking follows a trial-and-error approach. It may attempt a value and only discover later that the value creates a conflict.

Forward checking is more intelligent because it:

1. Assigns a value.
2. Examines neighboring unassigned variables.
3. Removes inconsistent domain values.
4. Detects impossible choices earlier.
5. Reduces unnecessary assignment attempts.
6. Reduces backtracking.
7. Reaches the solution more efficiently.

Therefore:

Forward Checking is more efficient than Plain Backtracking\boxed{\text{Forward Checking is more efficient than Plain Backtracking}}

for this CSP.

Final Conclusion

The university examination scheduling problem can be effectively modeled as a Constraint Satisfaction Problem.

The CSP contains:

5 Variables\boxed{5\ Variables} 3 Domain Values\boxed{3\ Domain\ Values}

and

7 Binary Constraints\boxed{7\ Binary\ Constraints}

A valid timetable obtained using both approaches is:

Slot	Courses
Slot 1	A – Artificial Intelligence, D – Operating Systems
Slot 2	B – Database Systems, E – Software Engineering
Slot 3	C – Computer Networks

Thus:

A=S1, B=S2, C=S3, D=S1, E=S2